

ATAVISHAALA

FARM MARKET · ENGINEERING

Team Execution Handbook

Architecture · Data Model · Engineering Rules · Week-by-Week Tasks
Phase 1 MVP · 4 Developers · 20 Working Days

BE-1	BE-2	MOB-1	FE-1
Platform · Catalog · Advisor Infra, CI/CD, schema, auth, users, addresses, catalog, search, media pipeline, Crop Advisor, SLA engine, notification transport, security, deploy	Commerce Cart, coupons, order FSM, inventory, Razorpay, webhooks, reconciliation, wallet ledger, referrals, cancellations, returns, reviews, support tickets	Mobile — React Native All ~40 screens across onboarding, catalog, product, cart, checkout, orders, tracking, Crop Advisor, profile, wallet, settings	Web Consoles Admin panel, CSV catalog importer, ops order board, expert Advisor console , coupon and wallet tools, review moderation, KPI dashboard

This is the **engineering half** of the Atavishaala delivery plan. Read §2.2 before writing any code — those twelve rules exist because breaking them is how zero-budget builds fail in week three. Milestones, risk register, cost audit and the release gate live in the companion volume, **Atavishaala — Manager's Strategic Roadmap**.

TEAM EXECUTION HANDBOOK

Architecture · engineering rules · data model · state machines · 4 weeks of tasks

§ 2.0 Architecture at a Glance

A **modular monolith**: one Node.js 20 + TypeScript service, one Postgres database, eight modules with enforced boundaries. Not microservices — with four developers and a four-week window, distributed tracing and inter-service contracts are a cost we cannot justify. Modules talk to each other through service functions, never by reaching into another module's tables.

The boundary discipline matters more than the deployment shape: when the Advisor eventually needs its own service, the seam is already drawn. A monolith with clean module boundaries can be split in a week; one without them cannot be split at all.

LAYER	CHOICE	WHY THIS ONE	TRADEOFF WE ACCEPT	TIER
Mobile	React Native + Expo SDK 53 (pinned)	One codebase, over-the-air updates for JS-only fixes, and Expo's managed native modules remove an entire class of Android build problems from a 4-week sprint.	Bare-workflow native modules need a prebuild; do not add one during the sprint.	Free
Web consoles	Vite + React SPA on Cloudflare Pages	Admin and expert consoles are internal tools behind a login — server rendering buys nothing. Cloudflare Pages allows commercial use on its free plan, which Vercel's Hobby tier does not.	No SEO and no SSR. Both irrelevant for an authenticated console.	Free
API	Node.js 20 + Express 5 + TypeScript, Zod at every boundary	The team already knows it. Zod schemas generate both the OpenAPI contract and runtime validation from one definition, which is how the frozen contract stays honest.	Single-threaded; CPU-heavy work must be pushed to jobs, not handled inline.	Free
Hosting	Google Cloud Run, asia-south1	Genuinely always-free at our volume, scales to zero, containerised, HTTPS and custom domains included. Railway's free tier no longer exists — do not follow the v2.0 handbook here.	Cold start ~1 s and no in-process state at all. Requires a billing account on file.	Always-free
Database	Neon serverless Postgres + Prisma	Postgres gives us the queue (SKIP LOCKED), full-text and trigram search, and JSONB snapshots. Neon's branching lets each developer test migrations against a real copy.	0.5 GB ceiling; autosuspend adds ~500 ms to the first query after idle.	Free tier
Search	pg_trgm GIN + tsvector on a generated column	A few thousand SKUs do not need a search engine. One index, no extra vendor, no sync job to drift out of date.	No typo tolerance beyond trigram similarity; no learned ranking.	Free
Media	Cloudinary, signed direct upload from client	Bytes never traverse our API, which is what keeps Cloud Run inside its free compute allowance. Automatic format and quality negotiation on images.	25 credits/month is a real ceiling; video must be capped and purged. See §2.6.	Free tier
Queue & jobs	Postgres job_queue + SKIP LOCKED, triggered by GitHub Actions cron	Removes Redis from the stack entirely. Jobs are rows — inspectable in SQL, individually re-runnable, and durable across a scale-to-zero restart.	Minimum granularity ~5 minutes. Not suitable for sub-second work.	Free

LAYER	CHOICE	WHY THIS ONE	TRADEOFF WE ACCEPT	TIER
Auth	Mobile OTP over WhatsApp Cloud API, SMS fallback; RS256 JWT + rotating refresh	WhatsApp is free within 1,000 conversations, is where farmers already are, and needs no DLT template approval to start testing.	Farmers without WhatsApp fall back to paid SMS. Meta template approval takes ~1 day.	Free tier
Notifications	FCM push + WhatsApp templates + Brevo email	FCM is free and unlimited. WhatsApp doubles as the advisory callback channel, so one integration serves two purposes.	Email is a courtesy channel only — farmers may not have one.	Free
Payments	Razorpay Standard Checkout + COD	Widest UPI coverage in India, webhooks are reliable, test mode is complete.	2% + GST per transaction. This is cost of goods, not infrastructure.	Pay per txn
Edge · DNS · SSL	Cloudflare Free	SSL, WAF, rate-limit rules, caching for the home feed, and DNS in one place at no cost.	Free-plan rate-limit rules are coarse. Application-level throttling still required.	Free
Observability	Sentry + PostHog + UptimeRobot + structured Pino logs	Errors, product funnels, uptime and request logs — four free tiers cover what a paid APM would.	No distributed tracing. A monolith makes that survivable.	Free tier

§ 2.1 Module Ownership & Dependency Matrix

Eight backend modules, each with exactly one owner. The secondary column is the person who reviews that module's pull requests and can pick it up if the owner is unavailable — read it as a cross-training obligation, not a formality. The dependency column is what must exist before work can start; check it before assigning anything.

MODULE	OWNER	REVIEW ER	MOBILE	WEB	HARD DEPENDENCY	WEEK
1 · Identity — OTP, JWT, sessions	BE-1	BE-2	MOB-1	FE-1	RS256 keypair in Secret Manager; WhatsApp template approved	W1
2 · Users — profile, language, addresses	BE-1	BE-2	MOB-1	FE-1	Schema part 1 (T1-04)	W1
3 · Catalog — suppliers, categories, products, variants, media, search	BE-1	BE-2	MOB-1	FE-1	Cloudinary signed preset (T2-01); GIN index (T2-03)	W2
4 · Crop Advisor — intake, media, assignment, SLA	BE-1	BE-2	MOB-1	FE-1	Media pipeline (T2-01); experts seeded (T2-14)	W2-W3
5 · Commerce — cart, coupons, pricing	BE-2	BE-1	MOB-1	—	Catalog variants and stock live (T2-04)	W3
6 · Orders — FSM, inventory, shipments, cancel, return	BE-2	BE-1	MOB-1	FE-1	Cart pricing engine (T3-01)	W3-W4
7 · Money — Razorpay, webhooks, wallet ledger, referrals	BE-2	BE-1	MOB-1	FE-1	Razorpay test keys; order FSM (T3-04)	W3-W4
8 · Platform — jobs, notifications, audit, config, i18n	BE-1	BE-2	MOB-1	FE-1	Job runner (T1-09); FCM service account	W1-W4

Boundary rule

A module owns its tables. Nothing outside modules/orders/ may write to orders, order_items or order_events — it calls ordersService.transition() instead. Cross-module reads go through an exported service function, never a raw query into someone else's table. This is enforced in code review, and it is the single discipline that decides whether this monolith can be split later.

Repository layout

PATH	CONTENTS
/apps/api	Express app. src/modules/{identity,users,catalog,advisor,commerce,orders,money,platform}/ each with routes.ts · service.ts · repo.ts · schema.ts. Shared middleware in src/http/, internal job handlers in src/jobs/.
/apps/mobile	Expo app. app/ for expo-router screens, features/ mirroring the API modules, i18n/{en,hi,te,kn}.json, api/ generated client.
/apps/console	Vite React SPA serving both the admin panel and the expert Advisor console, separated by role guard rather than by deployment.
/packages/contract	Zod schemas, generated OpenAPI document and the generated TypeScript client. The single source of truth for every request and response shape.
/packages/shared	Money helpers (paise integers), the order transition table, error codes, business-hours SLA calculator, referral constants.
/infra	Dockerfile, Cloud Run service YAML, Prisma migrations, seed scripts, .github/workflows/ including every cron.

§ 2.2 Twelve Non-Negotiable Engineering Rules

Read these before the first commit. Each one exists because violating it is a known, specific way this class of system fails — usually silently, usually with money involved, usually in the week you can least afford it. A pull request that breaks one of these is rejected on sight, without discussion about deadlines.

#	RULE	WHY — THE FAILURE IT PREVENTS	HOW IT IS CHECKED
1	All money is an integer in paise. No floats, no decimals, no strings, anywhere — database, API, or client.	Floating-point arithmetic on currency produces ₹0.01 discrepancies that make a wallet ledger impossible to reconcile and impossible to explain to a farmer.	Every money column is BIGINT named *_paise. Lint rule bans Number on price fields.
2	The wallet ledger is append-only. Never UPDATE or DELETE a ledger row. Balance is derived and cached, never authoritative.	A wallet whose balance cannot be recomputed from its own history is unauditable. When a farmer disputes ₹100, the ledger is the only answer you have.	Postgres rule blocks UPDATE/DELETE on wallet_ledger. A test asserts sum-of-rows equals cached balance across 200 transactions.
3	Every money-moving operation carries an idempotency key. Wallet credits, debits, refunds, referral payouts, webhook handling.	Networks retry. Webhooks arrive twice. Users double-tap. Without a unique key, a retry becomes a second ₹500 credit and there is no way to tell it from a legitimate one.	UNIQUE on wallet_ledger.idempotency_key and webhook_events.event_id. Replay test in CI.
4	Client callbacks never mutate state. Razorpay's browser callback updates nothing; only the verified server webhook does.	The client callback is attacker-controlled. Trusting it means anyone can mark an order paid by replaying a request.	The success handler only navigates and polls. Code review rejects any state change in a client payment handler.
5	Nothing lives in the server process. No node-cron, no setInterval, no in-memory cache, session or queue.	Cloud Run scales to zero and restarts freely. In-process state does not survive, so a timer that never fires becomes a silent, invisible bug.	Banned imports fail the lint step. All schedules are GitHub Actions crons calling /internal/jobs/:name.
6	Media bytes never traverse the API. Clients upload directly to Cloudinary with a short-lived signature; we store only the public ID.	Proxying a 2 MB video through Cloud Run burns free-tier compute and turns an upload into a request-timeout risk on a rural 4G connection.	Network inspector check is part of the Week 2 gate. No multipart file route exists in the API at all.
7	The server recomputes every price. Client-supplied amounts, discounts and totals are read only as a checksum to reject on mismatch.	Any client value is an attacker value. A tampered request otherwise buys a ₹4,000 sprayer for ₹1.	A tampered-request test in CI. The order creation endpoint's Zod schema does not accept a total at all.
8	Order status changes only through assertTransition(). No direct writes to orders.status, ever.	Ad-hoc status writes let an order jump from CONFIRMED to DELIVERED without a shipment, and the farmer's timeline becomes fiction.	Transition table in /packages/shared. Repository layer refuses a status write outside the service.
9	Stock is decremented conditionally, in one statement. UPDATE ... SET stock = stock - \$1 WHERE id = \$2 AND stock >= \$1.	Read-then-write oversells the last bag of fertilizer to four farmers at once. A conditional update is atomic without holding a lock.	Concurrency test: 10 simultaneous orders for 1 unit → exactly 1 success, 9 conflicts.
10	Personal data is redacted in logs. Phone, address, coordinates, payment identifiers and OTP never reach a log line or Sentry breadcrumb.	Logs get shared, exported and pasted into tickets. A phone number in a log is a data breach with extra steps.	Pino redaction paths configured centrally. Sentry beforeSend strips known keys. Grep audit in Week 4.
11	The OpenAPI contract is frozen after Day 3 of Week 1. Changes are pull requests against /packages/contract, merged on Mondays only.	Two clients and one API with three developers guessing at shapes is the most reliable way to lose a week to integration in a four-week sprint.	Clients import generated types; they cannot compile against a shape that does not exist.

#	RULE	WHY — THE FAILURE IT PREVENTS	HOW IT IS CHECKED
12	Every schema change is a committed migration. No manual SQL against the shared database, no exceptions for "just one column".	One undocumented column and nobody can rebuild the database, onboard a laptop, or trust that staging matches production.	CI runs migrations from empty on every pull request. A drift check fails the build.

§ 2.3 Data Model — 35 Tables by Module

The whole schema is deployed in Week 1, before any feature is built on it. A half-built schema is the most expensive thing to change in week three, because by then four people have written code against it. Estimated footprint at MVP scale is 40-60 MB — media stays in Cloudinary and only public IDs are stored here.

MODULE	TABLE	KEY COLUMNS & CONSTRAINTS	NOTES FOR IMPLEMENTERS
Identity	users	id · phone UNIQUE · name · role · language · district · state · referral_code UNIQUE · referred_by FK · status · created_at	Phone is the identity. No email, no password. referral_code generated at signup from name + 4 random chars, e.g. JAGA100.
	otp_codes	phone · code_hash · channel (WHATSAPP SMS) · attempts · expires_at · consumed_at · INDEX (phone, expires_at)	Store bcrypt hash, never the digits. Max 5 verify attempts, then invalidate. Purged hourly.
	refresh_tokens	user_id FK · token_hash · family_id · revoked_at · expires_at	Rotation with family revoke on reuse outside a 30 s grace window.
Users	addresses	user_id FK · label · line1 · line2 · village · district · state · pincode · lat · lng · is_default	lat/lng are what the 200 m delivery geofence measures against. Captured once at address creation.
	devices	user_id FK · fcm_token UNIQUE · platform · app_version · last_seen_at · active	Mark inactive on the first FCM send failure rather than retrying a dead token forever.
Catalog	suppliers	name · slug UNIQUE · logo_url · gst_number · verified · status	Drives the "Shop by Company" strip. Six to ten rows in Phase 1, entered by ops.
	categories	slug UNIQUE · parent_id · icon_url · sort_order · name_en/hi/te/kn	Translations live in columns, not a join table — four fixed languages, no runtime flexibility needed.
	products	supplier_id · category_id · name · brand · sku UNIQUE · description · usage_instructions · crop_tags[] · youtube_id · rating_avg · rating_count · status · search_text GENERATED · version	search_text is a stored generated column concatenating name, brand and category. Demo video is a YouTube ID, not a hosted file.
	product_variants	product_id FK · pack_label · pack_value · unit · mrp_paise · price_paise · stock_qty · is_default	One row per pack size (100 g / 250 g / 500 g / 1 kg). Cart and orders reference the variant, never the product.
	product_media	product_id FK · kind · cloudinary_public_id · sort_order	Images only. Video is the YouTube ID on the product row.
	banners	placement · title · subtitle · image_url · deep_link · active · sort_order	Every promotional strip in the mockups — the expert-advice banner, section banners — is a row, not a hardcoded asset.
	wishlist_items	user_id · product_id · UNIQUE(user_id, product_id)	Item 4 on the cut list.
	reviews	order_item_id UNIQUE FK · product_id · user_id · rating · body · status · created_at	UNIQUE on order_item_id is the entire anti-spam design: one review per delivered item, so only buyers can review and only once.
Advisor	consultations	ticket_no UNIQUE · user_id · crop_name · issue_text · language · status · sla_due_at · assigned_expert_id · created_at	sla_due_at is computed once at insert on business hours. See §2.6.
	consultation_media	consultation_id FK · kind · cloudinary_public_id · bytes · duration_s · sort_order · purge_after	purge_after is what keeps Cloudinary flat. Set to resolution date + 30 days.
	consultation_notes	consultation_id FK · expert_id · advice_text · recommended_variant_ids[] · channel · call_outcome · created_at	recommended_variant_ids is the commercial loop: advice becomes a deep link into the catalog.
	experts	user_id FK · name · phone · specialisations[] · languages[] · districts[] · active · current_load	Assignment matches language and district, then picks lowest load. Seeded by ops in Week 2.

MODULE	TABLE	KEY COLUMNS & CONSTRAINTS	NOTES FOR IMPLEMENTERS
Commerce	carts / cart_items	user_id UNIQUE · variant_id · qty · UNIQUE(cart_id, variant_id)	Server-side cart so it survives reinstall. Adding an existing variant increments quantity.
	coupons	code UNIQUE · kind (FLAT PERCENT REFERRAL) · value · min_order_paise · max_discount_paise · per_user_limit · global_limit · used_count · window · active	The referral coupon is one row of this table, which is why referrals ship even if the wider coupon engine is cut.
	coupon_redemptions	coupon_id · user_id · order_id · UNIQUE(coupon_id, user_id)	Enforces per-user limits at the database level rather than in application logic.
Orders	orders	order_no UNIQUE · user_id · address_snapshot JSONB · status · subtotal_paise · discount_paise · wallet_paise · shipping_paise · cod_fee_paise · total_paise · payment_mode · payment_status · coupon_id · version	Snapshot the address as JSONB. If the farmer later edits their address, historical orders must not silently change where they were shipped.
	order_items	order_id FK · variant_id · product_snapshot JSONB · qty · unit_price_paise · line_total_paise · review_prompt_sent_at	Snapshot name, brand, pack and image. Product edits and price changes must never rewrite history.
	order_events	order_id FK · from_status · to_status · actor_type · actor_id · note · lat · lng · created_at	Append-only, and the sole source for the mobile tracking timeline. Never derive the timeline from a duplicated status field.
	shipments	order_id FK · agent_id · otp_hash · delivered_lat/lng · distance_m · proof_media[] · delivered_at	distance_m is stored, not just checked — it is the audit trail for a disputed delivery.
	cancellations	order_id FK · reason_code · note · requested_by · refund_paise · status	Allowed from PENDING_PAYMENT, CONFIRMED and PACKED only. Reason codes match the mockup's list.
	returns	order_item_id FK · reason_code · note · photo_media[] · status · refund_paise · decided_by · decided_at	Per item, and deliberately does not change order status — the order stays DELIVERED. Photo evidence is mandatory.
Money	payments	order_id FK · gateway_order_id · gateway_payment_id · amount_paise · status · method · raw JSONB	One row per attempt, not per order. Keep the raw gateway payload for disputes.
	webhook_events	gateway · event_id UNIQUE · payload JSONB · processed_at · attempts	The unique index is the whole idempotency mechanism. Insert first, process second.
	wallet_accounts	user_id UNIQUE · balance_paise · version	A cache of the ledger. Updated only inside the same transaction that inserts a ledger row.
	wallet_ledger	wallet_id FK · direction · amount_paise · balance_after_paise · reason · ref_type · ref_id · idempotency_key UNIQUE · created_at	Append-only, enforced by a Postgres rule. balance_after_paise makes any historical balance readable without replaying the whole ledger.
	referrals	referrer_id · referee_id UNIQUE · code_used · referee_order_id · status · credited_at	UNIQUE on referee means one person can only ever be referred once, by anyone.
Platform	job_queue	kind · payload JSONB · run_after · attempts · max_attempts · locked_at · locked_by · done_at · last_error	Claimed with FOR UPDATE SKIP LOCKED. This is the Redis replacement.
	notifications	user_id FK · kind · title · body · deep_link · sent_at · read_at	Written before dispatch so the in-app notification list works even when a push fails.
	support_tickets / feedback_messages / faqs	user_id · order_id · category · subject · message · media[] · status	Help Centre and Write-to-Us. FAQs are rows so ops can edit copy without a release.
	audit_logs	actor_id · action · entity_type · entity_id · before JSONB · after JSONB · ip · created_at	Written for every console action — price edits, refunds, wallet adjustments, review moderation.
	app_config	key PK · value JSONB	Referral amounts, COD ceiling, free-delivery threshold, SLA hours, feature flags. Changing a business rule must not require a deployment.

§ 2.4 The Order State Machine

The farmer sees five states. The system has nine, because payment and cancellation need somewhere to live. Every write to `orders.status` goes through one function — `assertTransition(from, to, actor)` — which reads the table below and throws a 422 on anything not listed. There is no second code path, no admin override that skips the check, and no direct UPDATE in a repository file.

FROM	→ ALLOWED TO	WHO MAY TRIGGER IT	SIDE EFFECTS INSIDE THE SAME TRANSACTION
DRAFT	PENDING_PAYMENT	Farmer — checkout	Reserve stock with the conditional decrement; create the Razorpay order; debit wallet if the farmer chose wallet payment; write <code>order_events</code> .
PENDING_PAYMENT	CONFIRMED	System — webhook only	Mark payment captured; redeem coupon; queue the confirmation push; queue the referral credit job if this is the referee's first order.
PENDING_PAYMENT	PAYMENT_FAILED	System — webhook or the 10-minute sweep	Release reserved stock; reverse any wallet debit with a fresh idempotency key; notify the farmer with a retry deep link.
DRAFT	CONFIRMED	Farmer — COD checkout	COD skips the gateway entirely. Enforce the COD ceiling from <code>app_config</code> before allowing it.
CONFIRMED	PACKED	Ops console	Stock is already committed; nothing to recompute. Push notification only.
PACKED	DISPATCHED	Ops console	Assign the delivery agent; generate the 6-digit delivery OTP and store only its hash; push.
DISPATCHED	OUT_FOR_DELIVERY	Delivery agent	Push notification. This is the last state the farmer can see before delivery.
OUT_FOR_DELIVERY	DELIVERED	Delivery agent	Verify the delivery OTP; check the 200 m geofence and store the measured distance; collect cash for COD; schedule the review prompt for +24 h.
CONFIRMED / PACKED	CANCELLED	Farmer or ops	Release stock; credit the full amount to the wallet ledger; reverse the coupon redemption; push.
PENDING_PAYMENT	CANCELLED	System — 30-minute expiry job	Release stock. Nothing was captured, so there is nothing to refund.
DELIVERED	— terminal —	Nobody	Returns are recorded against <code>order_items</code> and never move the order backwards. A returned item is a refund event, not a status change.

ILLEGAL SKIPS ARE A TEST, NOT A COMMENT

The integration suite asserts that `CONFIRMED → DELIVERED` returns 422, that `PACKED → CANCELLED` by a delivery agent returns 403, and that `DELIVERED` is terminal. If those three tests pass, the FSM is real. Task T3-03.

WHY STOCK MOVES AT CHECKOUT, NOT AT CONFIRM

If stock were only committed on payment success, two farmers could both check out the last bag of seed and one would get a captured payment against zero inventory — a refund, an angry call, and a manual reconciliation. Reserving early and releasing on failure is cheaper.

The conditional decrement, verbatim

This is the entire concurrency story for inventory. No SELECT then UPDATE, no application-level lock, no advisory lock, no read of the current stock at all. If zero rows come back, somebody else won the race and we return 409 with the variant name.

```
UPDATE product_variants
  SET stock_qty = stock_qty - $2
 WHERE id = $1 AND stock_qty >= $2
 RETURNING stock_qty;
```

§ 2.5 Wallet, Refunds & Referral Settlement

The wallet is closed-loop store credit. Money can enter it and be spent inside Atavishaala, and it cannot be withdrawn to a bank account — that would make us a prepaid payment instrument issuer and require RBI authorisation. Treat that as a fixed boundary, not a backlog item.

Every balance change is an insert into `wallet_ledger` carrying an `idempotency_key`, inside the same transaction that updates the cached `wallet_accounts.balance_paise`. Nothing ever issues a bare `UPDATE` against a balance. If the two disagree, the ledger is right and the cache is rebuilt.

EVENT	DIRECTION	AMOUNT	IDEMPOTENCY KEY	GUARD THAT MAKES IT SAFE
Order cancelled before dispatch	CREDIT	Full amount paid	cancel:<order_id>	Unique key means a double-tap on Cancel credits once. The second insert fails and is swallowed as a success.
Return approved by ops	CREDIT	Item line total	return:<return_id>	Keyed on the return row, so re-approving an already-decided return is a no-op.
Payment failed after wallet debit	CREDIT	The debited amount	revpay:<payment_id>	Reversal is a new row with its own key, never a delete or an edit of the original debit.
Referral bonus to the referrer	CREDIT	₹100	ref:<referral_id>	Fires only when the referee's first order reaches DELIVERED, from the queued job — never from the mobile client.
Wallet used to pay for an order	DEBIT	Order total	pay:<order_id>	Balance re-read inside the transaction. Insufficient balance is a 409 before the order is created.
Ops goodwill adjustment	CREDIT	Ops-entered	adj:<audit_log_id>	Requires a reason string, writes to <code>audit_logs</code> , and is capped per day per operator.
Farmer adds money · CUT LIST #7	CREDIT	Top-up amount	topup:<payment_id>	Deferred. Nothing in the MVP needs it — refunds and referrals fill the wallet on their own.

Payment composition — deliberately not split-tender

An order is paid **either** entirely from the wallet **or** entirely through Razorpay or COD. Splitting a payment across the wallet and the gateway sounds like a small feature and is not: every failure, refund, cancellation and return then has to apportion money back across two rails correctly, and each of those is a place to lose rupees. Wallet-or-gateway is one line of validation and removes an entire class of bug from the MVP. Revisit in Phase 2 when there is a real ledger reconciliation report to check it against.

Referral flow, end to end

STEP	TRIGGER	WHAT HAPPENS	WHERE
1	Referee signs up	Enters a code on the personal-info screen. Validate it exists, is active, and is not the referee's own. Write <code>users.referred_by</code> and a <code>referrals</code> row with status PENDING.	T2-04 · mobile + identity
2	Referee checks out for the first time	The referral coupon is auto-applied — ₹50 off, subject to the minimum order value in <code>app_config</code> . It is an ordinary coupon row, so it flows through the same discount and redemption code as everything else.	T3-06 · commerce
3	That order reaches DELIVERED	The FSM transition enqueues <code>referral.credit</code> . Do not credit on order creation — a cancelled order would have paid out ₹100 for nothing.	T3-04 · orders
4	Job runs	Re-check that the order is still DELIVERED and not returned in full, then insert a ₹100 CREDIT keyed <code>ref:<referral_id></code> , set the referral to CREDITED, and push "₹100 added to your wallet".	T3-11 · money

STEP	TRIGGER	WHAT HAPPENS	WHERE
5	Fraud brake	One referral per referee, enforced by the unique index. Cap credited referrals per referrer per month from config. Reject codes when the referee's phone matches the referrer's. Ops sees a referral leaderboard and can freeze a code.	T3-14 · console

§ 2.6 CROP ADVISOR — THE PART THAT MUST NOT BE COMPROMISED

Two things in this module are genuinely hard on a zero budget: moving up to five photos and three videos per ticket without paying for bandwidth or transcoding, and guaranteeing a two-hour callback when nothing is watching the clock. Both are solved below. Neither is on the cut list.

The media budget, and why the caps are what they are

Cloudinary's free plan gives 25 monthly credits, where one credit is roughly 1 GB of storage, 1 GB of delivery, or 1,000 image transformations — and **video transformations burn credits far faster than images**. Server-side transcoding of farmer video is therefore not affordable, and no free alternative changes that arithmetic. The answer is to make the client produce a file that needs no processing at all, and to delete it once it has served its purpose.

CONTROL	SETTING	ENFORCED WHERE	WHAT IT BUYS US
Client-side compression	react-native-compressor, 720p, 1.2 Mbps target	Mobile, before upload	A 15-second field clip lands around 2–3 MB instead of 40 MB. This single control is worth more than every other optimisation combined.
Hard duration cap	15 seconds per clip, 3 clips maximum	Mobile picker + server validation	Bounds the worst case per ticket at roughly 9 MB of video. Farmers photograph a leaf; they do not need a minute of footage.
Photo caps	5 photos, resized to 1600 px longest edge, JPEG q80	Mobile, then eager thumbnail on upload	About 400 KB each. One eager 400 px thumbnail per photo for the expert console list view.
Server byte ceiling	Reject any asset over 8 MB	Signed-upload params + post-upload check	A patched or rooted client cannot bypass the cap, because the signature encodes max_bytes.
Direct signed upload	Client → Cloudinary, never through the API	POST /advisor/media/sign	Media bytes never touch Cloud Run, so uploads cannot consume request time, memory or the free invocation budget.
No video transformation	Store and stream the original; poster frame only	Upload preset — eager transforms disabled for video	Video credits are spent on delivery alone. The expert console streams it once and that is the entire lifecycle.
30-day purge	purge_after = resolved_at + 30 days	Nightly media.purge job	Storage stays flat month over month instead of compounding. This is what keeps the plan free in month six rather than only in month one.
Upload gate	Ticket must exist and be OPEN; 3 tickets per user per day	Advisor service	Stops a bored or malicious user from turning the Advisor into free cloud storage.

THE ARITHMETIC, STATED PLAINLY

At 150 tickets a month averaging 3 photos and 1 clip, that is roughly 0.6 GB stored and a similar amount delivered — well inside 25 credits. The plan breaks somewhere past **400 tickets a month** with video on most of them. Watch the credit meter weekly, not monthly.

THE DEGRADATION, PRE-AGREED

If credits pass 70%, video intake switches to **one clip of 10 seconds** via a config flag, and the purge window drops from 30 days to 7. Photos are never restricted — a photo of a diseased leaf is the minimum viable input for the expert.

The SLA clock — two hours of business time, not wall time

A ticket raised at 19:30 must not be marked breached at 21:30, when no expert is working. `sla_due_at` is computed once at insert, walking forward through the business window (08:00–20:00 IST, seven days a week — farming does not take weekends) and rolling over into the next morning if the two hours do not fit. Storing the result rather than computing it on read means the countdown the farmer sees, the expert sees, and the escalation job sees are the same number, always.

RAISED AT	sla_due_at BECOMES	WHY
09:15	11:15 same day	Fully inside the window. The simple case.
19:30	09:30 next day	30 minutes consumed before close, the remaining 90 carried to 08:00.
22:40	10:00 next day	Raised outside hours. The clock starts at 08:00, not at submission.
06:00	10:00 same day	Early-morning submissions — common among farmers — start at open.

Escalation ladder

A GitHub Actions cron hits POST /internal/jobs/advisor.sla every 15 minutes. It runs four checks in one query pass and is the only thing that decides a breach — no timers, no in-memory schedules, nothing that dies when the container scales to zero:

WHEN	ACTION	WHO IS TOLD
T + 0 min	Ticket assigned to an expert by matching language, then district, then lowest current_load. Unassignable tickets go to an ops queue immediately.	Expert — FCM + WhatsApp
T + 45 min	First nudge if the expert has not opened the ticket.	Expert
T + 90 min	Reassign to the next available expert and mark the ticket AT_RISK on the console, sorted to the top in red.	Expert + ops
T + 120 min	Mark BREACHED, write an audit_logs row, and send the farmer an honest "we are running late, an expert will call you shortly" message rather than silence.	Ops + the farmer

THE HONEST DEPENDENCY

The two-hour SLA is an **operational** promise the software can only measure. It holds if a human agronomist is reachable during business hours. Confirm expert staffing and rostering before the number appears anywhere a farmer can read it.

WHAT THE FARMER SEES

A ticket number, a live countdown, the assigned expert's name, and after the call the written advice plus tappable product recommendations. Crop Advisory History in the profile is the same list, read-only.

§ 2.7 Week 1 — Foundation, Contract & Identity

WEEK 1 · FOUNDATION, CONTRACT & IDENTITY

Focus: a deployed, monitored, migrating API that a real phone can log into

Nothing in Week 1 is a feature, and that is the point. By Friday a farmer's actual phone number receives an OTP from the deployed API, the whole schema is migrated, the job runner is executing on a schedule, and every compliance clock outside our control has already started ticking. Week 1 is the week that decides whether Weeks 2 to 4 are calm.

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T1-01	BE-1	Platform · Monorepo & toolchain	1. Create the repo with pnpm workspaces: /apps/api, /apps/mobile, /apps/console, /packages/contract, /packages/shared, /infra. 2. Pin Node 20 LTS in .nvmrc and engines; pin Expo SDK 53. No version drifts mid-sprint. 3. Add TypeScript strict, ESLint, Prettier, Husky pre-commit running lint plus typecheck. 4. Create the empty module folders from §2.1 with routes/service/repo/schema stubs so nobody invents their own layout. 5. Write README.md with local setup in under ten commands and a .env.example covering every variable in Appendix A.	✓ pnpm i & pnpm dev works from a clean clone on all four machines ✓ Commit blocked locally on a lint or type error ✓ Branch convention live: feat/<module>-<short>, squash merge to main
T1-02	ALL	Platform · API contract — frozen Day 3	1. Two half-day workshops: walk every screen in the mockups and name the endpoint that feeds it. 2. Write Zod schemas in /packages/contract for every request and response; generate the OpenAPI document from them so there is one source of truth. 3. Agree the error envelope once — { code, message, details? } — and the pagination shape: cursor, not offset. 4. Stand up a Prism mock server from the OpenAPI file so MOB-1 and FE-1 stop waiting on BE work. 5. Freeze at end of Day 3. After that, changes need a PR to the contract package plus a note in the Wednesday sync.	✓ @atavishaala/contract published to the workspace and imported by API, mobile and console ✓ Mock server URL in every developer's env — mobile and console build against it from Day 4 ✓ Contract diff is visible in git, so no silent field renames
T1-03	BE-1	Platform · Hosting, database, CI/CD	1. Create the Google Cloud project, enable Cloud Run and Secret Manager, and deploy a hello-world container to prove the path end to end. 2. Create the Neon project with main and preview branches; store the pooled connection string in Secret Manager. 3. Write the GitHub Actions pipeline: typecheck → test → build image → push to Artifact Registry → deploy to Cloud Run → run prisma migrate deploy. 4. Set min-instances to 0, concurrency 80, memory 512 MB, and a request timeout of 60 s. 5. Register the domain and put Cloudflare in front for DNS, SSL and WAF; map api. to Cloud Run.	✓ Merge to main deploys automatically in under six minutes with no manual step ✓ https://api.<domain>/healthz returns 200 over TLS ✓ Rollback is a single re-deploy of the previous revision, and it has been tested once

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T1-04	BE-1	Data · Schema part 1 + migrations	1. Model identity, users, addresses, devices and the platform tables from §2.3 in schema.prisma. 2. Money columns are BigInt parse. No Float, no Decimal, no rupees. Add a CI grep that fails the build on a Float in a money field. 3. Add the Postgres rule that makes wallet_ledger reject UPDATE and DELETE, even though the table arrives in T1-08. 4. Test the migration on the Neon preview branch first, then apply to main. 5. Write seed.ts with two test farmers, one expert and one admin.	✓ Migration applied to both branches; prisma migrate status clean ✓ Append-only rule proven by a test that expects the UPDATE to throw ✓ Any developer can reset and reseed a local database in one command
T1-05	BE-1	Identity · OTP request & verify	1. POST /auth/otp/request — normalise the number to E.164, store a bcrypt hash of a 6-digit code with a 5-minute expiry. 2. Send via WhatsApp Cloud API as the primary channel; fall back to 2Factor.in SMS on any non-2xx, and record which channel was used. 3. Throttle by phone and by IP: 3 requests per 10 minutes, 10 per day. Return 429 with retry_after. 4. POST /auth/otp/verify — max 5 attempts, then invalidate the code entirely rather than letting it be brute-forced. 5. Return is_new_user so the app knows whether to route to personal info or straight to the home screen.	✓ A real phone receives the OTP over WhatsApp; killing the WhatsApp credential falls back to SMS ✓ Codes never appear in logs or in Sentry — verified by reading the deployed log output ✓ Throttle proven by a script that fires four requests and gets a 429 on the fourth
T1-06	BE-1	Identity · JWT & refresh rotation	1. Generate an RS256 keypair, keep the private key in Secret Manager, and expose the public JWKS. 2. Access token 15 minutes; refresh token 60 days, stored as a hash with a family_id. 3. On refresh, rotate and revoke the old token. Reuse of a rotated token outside a 30-second grace window revokes the entire family and forces re-login. 4. Write the auth middleware with a role guard: farmer, expert, ops, admin. 5. Add POST /auth/logout revoking the family, and log-out-everywhere for Account Settings.	✓ Token replay test passes: reused refresh token returns 401 and kills the family ✓ Role guard unit-tested for all four roles against a protected route ✓ Concurrent refresh from two app instances does not log the user out
T1-07	BE-1	Users · Profile, language, addresses	1. GET/PATCH /me for name, language and district — the personal-info screen writes here. 2. Address CRUD with a default-address invariant enforced in a transaction, so exactly one address is default at all times. 3. Capture lat/lng at address creation; it is what the delivery geofence in T4-04 measures against. 4. Validate pincode against a seeded serviceable-pincode list and return a clear "we do not deliver here yet" message. 5. POST /devices to register the FCM token on every app launch, upserting on the token.	✓ Personal-info and Saved Address screens fully functional against the real API ✓ Deleting the default address promotes another one instead of leaving none ✓ Unserviceable pincode blocks checkout early rather than at payment

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T1-08	BE-2	Data · Schema part 2 + fixtures	1. Model catalog, commerce, orders, money and advisor tables from §2.3, including the generated search_text column. 2. Add every unique constraint that carries a business rule: webhook_events.event_id, wallet_ledger.idempotency_key, reviews.order_item_id, referrals.referee_id. 3. Add the indexes the read paths need now, not after the first slow query: orders by user and status, consultations by sla_due_at, job_queue by kind and run_after. 4. Seed 40 realistic products across 6 suppliers with variants, stock and Telugu and Hindi category names — MOB-1 needs believable data, not "Test Product 1". 5. Seed app_config: referral amounts, COD ceiling, free-delivery threshold, SLA hours, feature flags.	✓ Full schema deployed in Week 1; no structural migration expected after Week 2 ✓ Seed produces a catalog that looks like the mockups on a real device ✓ Constraint tests prove each unique index rejects the duplicate it exists to prevent
T1-09	BE-1	Platform · Job queue & scheduler	1. Create job_queue and a worker that claims rows with SELECT ... FOR UPDATE SKIP LOCKED LIMIT 10. 2. Expose POST /internal/jobs/:name guarded by a shared secret header and never routed through the public API surface. 3. Add GitHub Actions cron workflows: advisor.sla every 15 min, payments.reconcile every 10 min, reviews.prompt hourly, media.purge and otp.cleanup nightly. 4. Exponential backoff with max_attempts 5; a dead job writes last_error and raises a Sentry event. 5. Make every handler idempotent and re-runnable — assume it will run twice, because eventually it will.	✓ A job survives the container scaling to zero and still runs on schedule ✓ Two concurrent workers never process the same row — proven by a 500-job load test ✓ Failed jobs are visible in Sentry with the payload, redacted of personal data
T1-10	BE-2	Platform · Notification transport	1. Wrap FCM in a single notify(userId, kind, payload) function; everything else calls only this. 2. Write the notifications row before dispatch so the in-app list is correct even when the push fails. 3. Add WhatsApp and SMS as alternate channels behind the same interface, chosen per notification kind from config. 4. Deactivate an FCM token on the first UNREGISTERED response rather than retrying a dead device forever. 5. Define all notification kinds up front with deep links: order confirmed, packed, dispatched, out for delivery, delivered, review prompt, advisor assigned, advisor calling, wallet credited, referral earned.	✓ A push arrives on a physical Android device and opens the right screen ✓ Notification list renders from the database with unread counts ✓ No feature module imports the FCM SDK directly
T1-11	BE-2	Platform · Vendor accounts & the compliance clock	1. Day 1, before any code: file TRAI DLT registration for the SMS sender ID and templates. It takes 3–7 working days and is entirely outside our control. 2. Submit the WhatsApp Cloud API OTP template for approval and complete Meta business verification. 3. Start Razorpay KYC, collect test keys the same day, and note that live keys can take a week. 4. Create Cloudfire, Sentry, PostHog, Brevo and UptimeRobot accounts on a shared team login, not a personal one. 5. Record every credential in Appendix A and load it into Secret Manager and GitHub Secrets.	✓ All approval clocks started on Day 1 — nothing blocks Week 4 waiting on a vendor ✓ Test keys available immediately so payments work end to end in Week 3 ✓ No account tied to one person's personal email

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T1-12	MOB-1	Mobile · App shell, tokens & i18n	1. Initialise Expo SDK 53 with expo-router, then build the tab and stack layout for the whole app before any screen has content. 2. Encode the brand palette and type scale as tokens once — green #009411, violet #8735F5, near-black text — so no screen hardcodes a colour. 3. Build the shared component set: Button, Input, Card, ProductCard, Sheet, EmptyState, SkeletonList, ErrorState, Toast. 4. Set up i18n with en/hi/te/kn JSON files and no literal strings in components — retrofitting four languages in Week 4 is a rewrite. 5. Wire TanStack Query against the mock server with a typed API client generated from the contract.	✓ Full navigation walkable with placeholder screens on Day 3 ✓ Switching device language changes the UI immediately ✓ A new screen takes minutes, because every primitive already exists
T1-13	MOB-1	Mobile · Onboarding & authentication flow	1. Landing screen, language picker, then the phone-number screen with a 10-digit validator and a +91 prefix. 2. OTP screen with 6-box auto-advance input, auto-read from the SMS on Android, a 30-second resend timer and a clear error state. 3. Personal-info screen for new users: name, district, state and the optional referral code, with inline validation of the code. 4. The three onboarding slides — Expert Crop Guidance, Trusted Agricultural Products, Track Every Order — with skip, and shown only once. 5. Route existing users straight to the home screen; branch on is_new_user from T1-05.	✓ A new number completes signup and lands on the home screen against the deployed API ✓ A returning number skips onboarding entirely ✓ Every error case has copy: wrong OTP, expired OTP, throttled, no network
T1-14	MOB-1	Mobile · Session & resilience layer	1. Store tokens in Expo SecureStore, never in AsyncStorage. 2. Add an Axios interceptor that refreshes on 401 and queues concurrent requests behind a single refresh so four parallel calls do not trigger four rotations. 3. Handle the refresh-family revoke case by clearing state and returning to the landing screen without a crash. 4. Add a global offline banner and retry, plus a Cloud Run cold-start friendly 20-second timeout with a skeleton rather than a spinner. 5. Install Sentry for React Native with release tagging on every EAS build.	✓ App relaunch after two days still has a valid session ✓ Aeroplane mode shows the offline state everywhere instead of blank screens ✓ First request after idle succeeds despite the cold start
T1-15	FE-1	Console · Shell, auth & deployment	1. Scaffold Vite plus React plus TypeScript with Tailwind and the same design tokens as mobile. 2. Build the app shell: sidebar, top bar, role-aware navigation, and a data-table component with cursor pagination, filters and CSV export used by every later screen. 3. Ops and admin log in with the same OTP endpoints, gated on role — no second auth system. 4. Deploy to Cloudflare Pages on push, with a preview URL per pull request. Not Vercel — the Hobby plan forbids commercial use. 5. Add a role-permission map in one file so a new screen declares who may see it.	✓ admin.<domain> live with TLS behind the ops login ✓ A farmer's token cannot reach the console — verified, not assumed ✓ The shared table component is what every subsequent console screen uses

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T1-16	FE-1	Console · Supplier, category & config tools	1. Supplier CRUD with logo upload through the signed-upload endpoint, GST field and a verified flag. 2. Category tree CRUD with drag ordering, icon upload and all four language names in one form. 3. app_config editor with typed inputs and a confirmation step, so changing the COD ceiling or the referral amount is not a deployment. 4. Audit-log viewer with before-and-after JSON diff, filterable by actor and entity. 5. Seed the six real suppliers and the category tree from the mockups with genuine copy.	✓ Ops can populate suppliers and categories without a developer ✓ Every console write appears in the audit log with a readable diff ✓ Business rules are editable at runtime and the change is logged
T1-17	ALL	Friday gate — Week 1 demo	1. Demo on deployed infrastructure only. Nothing runs on a laptop. 2. Prove it: log in with a real phone over WhatsApp OTP, save an address, see a seeded catalog in the console, and watch a cron job execute. 3. Confirm the contract is frozen and the mock server matches the deployed API. 4. Review the free-tier meters for the first time and record the numbers. 5. Re-read the \$1.6 cut list together and confirm nobody has quietly started building a cut item.	✓ Gate passed only if a phone that has never seen the app can log in ✓ Free-tier baseline recorded in the shared tracking sheet ✓ Week 2 assignments confirmed with no unresolved blockers

§ 2.8 Week 2 — Catalog, Media Pipeline & Crop Advisor Intake

WEEK 2 · CATALOG, MEDIA & ADVISOR INTAKE

Focus: a browsable catalog on a phone and a consultation ticket that reaches an expert

Week 2 delivers the two pipelines everything else hangs from: the catalog read path that four mobile screens consume, and the signed media pipeline that both product images and Advisor evidence share. The Advisor is started in Week 2 rather than Week 3 on purpose — it is the differentiator, so it gets the calm week, not the compressed one.

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T2-01	BE-1	Platform · Signed media pipeline	1. POST /media/sign returns a Cloudinary signature scoped to a folder, resource type, allowed formats, max_bytes and an expiry. Bytes go client → Cloudinary and never through the API. 2. Create upload presets per purpose: product (eager 400 px and 800 px thumbnails), advisor_photo (one 400 px eager), advisor_video (no transformations at all). 3. POST /media/confirm verifies the asset exists, reads its real bytes and duration back from Cloudinary, and rejects anything over the cap — a patched client cannot lie. 4. Serve every read through f_auto,q_auto plus explicit width so no original image is ever delivered to a phone. 5. Write the media.purge job against purge_after and a Cloudinary usage probe that logs credits daily.	✓ Upload works from mobile and console with no bytes through Cloud Run ✓ A forged 40 MB upload is rejected at confirm and the orphan is deleted ✓ Daily credit usage visible as a number, not a guess
T2-02	BE-1	Catalog · Suppliers, categories, banners	1. GET /suppliers and GET /categories returning the tree with localised names chosen by the Accept-Language header. 2. GET /banners?placement= for every promotional strip in the mockups, including the expert-advice banner, each with a deep link. 3. Add HTTP caching: Cache-Control plus ETag on all catalog reads, and let Cloudflare serve the repeats. 4. Return only active rows, sorted by sort_order, so ops controls presentation without a release. 5. Add an in-process 60-second memo cache for the category tree — it changes weekly and is read on every home screen load.	✓ Home screen strips render from the API, not from hardcoded arrays ✓ Repeat catalog loads are served from the Cloudflare edge ✓ Switching app language changes category names
T2-03	BE-1	Catalog · Search without a search engine	1. Enable pg_trgm; add a GIN index on the generated search_text column. 2. Add a tsvector index for word-prefix matching, and combine the two: full-text first, trigram similarity as the typo-tolerant fallback. 3. GET /search?q=&category=&supplier=&crop=&sort=&cursor= with cursor pagination, never offset. 4. Rank by a weighted score of text similarity, in-stock status and rating so out-of-stock items sink instead of leading. 5. Seed 20 realistic vernacular and misspelt queries as a fixture test — "uria", "kirmi nashak", "tomato seed" — and assert the right product is in the top three.	✓ Search returns in under 150 ms on the seeded catalog ✓ Typos and partial words still find the product ✓ No Elasticsearch, no Algolia, no second vendor and no monthly bill

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T2-04	BE-1	Catalog · Products, variants & stock	1. GET /products list with filters, and GET /products/:slug returning variants, media, YouTube ID, usage instructions, rating summary and the top three reviews in one call. 2. Expose stock as a band — In stock, Low stock, Out of stock — rather than a raw count, so competitors cannot read inventory. 3. Return mrp_paise and price_paise separately; the discount badge is computed by the client from those two numbers. 4. Admin write endpoints for products and variants with optimistic concurrency on version, so two operators cannot silently overwrite each other. 5. Ship the conditional stock decrement and release helpers now, with tests, so Week 3 checkout only has to call them.	✓ Product detail screen renders fully from a single request ✓ Stock decrement proven atomic by 50 parallel requests for the last 10 units ✓ Price is a server number everywhere; the client never computes a total
T2-05	BE-1	Catalog · Home feed & recommendations	1. GET /home composes the entire home screen in one response: categories, banners, supplier strip, and the Seeds, Fertiliser and Pesticide sections. 2. Recommended-for-you is a SQL query over the farmer's delivered order history — same category, same crop tags, excluding what they already bought this month. No machine learning. 3. Cold start for new users falls back to best-sellers in their district, then to global best-sellers. 4. Cache the shared portion for 5 minutes; only the personalised block is computed per request. 5. This is cut-list item #1 — keep it behind a config flag so it can be switched to best-sellers in one edit if Week 4 gets tight.	✓ Home screen is a single request, so it loads fast on a rural connection ✓ New users see a populated screen rather than empty sections ✓ Feature flag proven to degrade cleanly
T2-06	BE-1	Advisor · Intake, media & SLA clock	1. POST /advisor/consultations takes crop name, free-text issue, language and preferred call window; returns a human ticket number such as ADV-2481. 2. Compute sla_due_at at insert using the business-hours walk in §2.6 and store it. Never compute it on read. 3. Attach media via POST /advisor/consultations/:id/media after /media/confirm; enforce 5 photos, 3 clips, 15 s and 8 MB server-side. 4. Rate-limit to 3 open tickets per farmer per day and require the ticket to be OPEN before media is accepted. 5. GET /advisor/consultations is Crop Advisory History in the profile — same endpoint, no second implementation.	✓ Ticket created with photos and one clip from a real device on mobile data ✓ SLA due time correct for a 19:30 submission — asserted in a unit test ✓ Caps cannot be exceeded even with a modified client
T2-07	BE-1	Advisor · Assignment & escalation	1. Assignment on insert: filter experts by language, then district, then pick the lowest current_load. Unassignable tickets land in an ops queue immediately rather than silently waiting. 2. Notify the expert on FCM and WhatsApp with the ticket link the moment it is assigned. 3. Implement the advisor.sla job with the four-step ladder from §2.6: 45-minute nudge, 90-minute reassign, 120-minute breach, farmer notified honestly. 4. POST /advisor/consultations/:id/notes for the expert to record advice, call outcome and recommended variant IDs. 5. Resolving a ticket sets purge_after on its media and pushes the written advice to the farmer.	✓ Ticket reaches an expert's phone within seconds of creation ✓ A deliberately ignored ticket escalates and breaches on schedule ✓ Expert advice appears in the farmer's history with tappable product links

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T2-08	BE-2	Commerce · Cart & wishlist	1. Server-side cart keyed to the user so it survives reinstall and device change. 2. POST /cart/items upserts on variant, incrementing quantity rather than creating a duplicate line. 3. GET /cart recomputes every price from the database on read and flags any line whose price changed or whose stock dropped — the client displays, it never calculates. 4. Cap quantity per line from app_config and validate against the stock band. 5. Wishlist add, remove and list, plus a move-to-cart action. This is cut-list item #4 — keep it in one file.	✓ Cart survives logout, reinstall and a second device ✓ A price change while items sit in the cart is surfaced, not silently applied ✓ Wishlist is isolated enough to delete in an afternoon if it is cut
T2-09	BE-2	Catalog · Reviews read & rating rollup	1. GET /products/:id/reviews with cursor pagination and a rating histogram. 2. Maintain rating_avg and rating_count incrementally in the same transaction as the review insert; do not recompute an average over the whole table on every read. 3. Moderation states — PENDING, PUBLISHED, REJECTED — with only published reviews visible publicly. 4. Add a profanity and phone-number filter that flags rather than blocks, so ops decides. 5. Write path lands in Week 4 with the review prompt; the read path exists now because product detail needs it.	✓ Product detail shows real reviews and a histogram ✓ Rating rollup stays correct under concurrent inserts ✓ Ops can unpublish a review and it disappears immediately
T2-10	BE-2	Support · Help centre, FAQs & write-to-us	1. FAQs as database rows in all four languages so ops edits copy without a release. 2. POST /support/tickets with an optional order reference, a category and up to 3 photos through the same signed-upload path. 3. Write-to-Us posts to the same table with a different category — one pipeline, two entry points. 4. Email ops through Brevo on every new ticket, and push the farmer when ops replies. 5. In-app help articles are cut-list item #3; the fallback is a phone number and a WhatsApp deep link, which is what most farmers will use anyway.	✓ Farmer can raise a ticket with a photo and see the reply in-app ✓ Ops is emailed within a minute of a ticket arriving ✓ Degrading to call-and-WhatsApp is a config flag, not a code change
T2-11	BE-2	Money · Razorpay spike & webhook receiver	1. Pulled forward from Week 3 deliberately — payment integration is where unknown unknowns live, and Week 2 has the slack to absorb them. 2. POST /payments/webhook verifies the HMAC signature before parsing the body, inserts into webhook_events on the unique event_id, returns 200, and processes asynchronously. 3. A duplicate delivery hits the unique index, is swallowed, and still returns 200 — Razorpay retries aggressively and must never see a 500. 4. Create a test order in Razorpay test mode and drive it through success, failure and abandonment. 5. Stand up the payments.reconcile job skeleton that polls the gateway for orders stuck in PENDING_PAYMENT over 10 minutes.	✓ Signature verification rejects a tampered payload ✓ The same webhook replayed 20 times produces exactly one state change ✓ No payment logic depends on the mobile client calling back

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T2-12	FE-1	Console · Product management & CSV import	1. Product CRUD with a variant sub-table, rich description, usage instructions, crop tags and the YouTube ID field. 2. CSV importer with a column mapper, a dry-run preview showing per-row errors, and a partial-commit mode — ops will onboard 200 SKUs and will not type them one at a time. 3. Bulk stock and price update by SKU, with every change written to audit_logs. 4. Low-stock dashboard with a configurable threshold, since running out of a fertiliser silently is a lost sale. 5. Surface the optimistic-concurrency conflict from T2-04 as a readable "someone else edited this" message.	✓ 200 SKUs imported in one pass with a row-level error report ✓ A price edit is traceable to a person and a timestamp ✓ No developer needed to launch a new supplier's range
T2-13	FE-1	Console · Media manager & banners	1. Drag-and-drop image upload to the signed endpoint with reordering, alt text and a delete that also removes the Cloudinary asset. 2. Banner scheduling by placement with a live preview at phone width, so ops sees what the farmer will see. 3. Show the Cloudinary credit meter on the dashboard with a red state past 70% — the person uploading images is the person who should see the cost. 4. Warn on any upload over 1 MB and offer to downscale in the browser before sending. 5. Orphan finder listing assets with no database row, with a bulk delete.	✓ Ops manages all imagery without touching Cloudinary directly ✓ Credit usage is visible to the people who generate it ✓ Storage stays clean instead of accumulating orphans
T2-14	FE-1	Console · Expert roster & Advisor console v1	1. Expert CRUD: name, phone, specialisations, languages, districts, active flag — seed the real roster this week, because the Advisor cannot be tested without it. 2. Advisor queue board sorted by sla_due_at ascending, with a live countdown per ticket and AT_RISK rows in red at the top. 3. Ticket detail: photo gallery with zoom, inline video player, the farmer's issue text, crop, district and language, plus one-tap call and WhatsApp links. 4. Advice form writing consultation_notes with a product picker feeding recommended_variant_ids, and a call-outcome dropdown including "did not answer". 5. Manual reassign with a reason, and an SLA-performance strip showing today's on-time percentage.	✓ An expert can work a full ticket from queue to resolution in this screen ✓ Breach risk is impossible to miss when the board is open ✓ Real experts loaded, so Week 3 tests the system with actual people
T2-15	MOB-1	Mobile · Home screen	1. Build the full home screen from GET /home: search bar, shop-by-category grid, the expert-advice banner, shop-by-companies strip, recommended-for-you, and the Seeds, Fertiliser and Pesticide sections. 2. Every strip is a horizontally scrolling FlashList of the shared ProductCard, so one component change updates the whole screen. 3. Skeleton loading, pull-to-refresh, and an error state with retry — this screen loads on a weak rural connection more often than not. 4. Cache the response with TanStack Query and a stale-while-revalidate window so a relaunch shows content instantly. 5. The expert-advice banner deep-links straight into Advisor intake; it is the most important tap on the screen.	✓ Home screen matches the mockup and is entirely data-driven ✓ Feels instant on relaunch thanks to the cache ✓ No crash and no blank screen when any section is empty

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T2-16	MOB-1	Mobile · Search, listing & filters	1. Search screen with 300 ms debounce, recent searches held locally, and an empty state that suggests categories. 2. Category and supplier listing screens sharing one grid component with infinite cursor scroll. 3. Filter sheet: price range, supplier, crop tag, in-stock-only; sort by relevance, price or rating. Filters live in the URL params so back navigation restores them. 4. Show applied filters as removable chips — farmers need to see why a list looks short. 5. Handle the slow first request from a cold container with a skeleton rather than a spinner.	✓ Search finds products including misspellings, end to end ✓ Scrolling 200 results stays smooth on a low-end Android device ✓ Back from a product returns to the same scroll position and filters
T2-17	MOB-1	Mobile · Product detail	1. Image carousel with page dots, then name, brand, supplier, price with a struck-through MRP and a computed discount badge. 2. Variant selector as pack-size chips that re-price the screen instantly from the already-fetched data. 3. Delivery-availability block using the default address pincode, with an address switcher. 4. Collapsible description and usage instructions, the YouTube demo video as a lazy embed, and the reviews block with a histogram and a see-all link. 5. Sticky bottom bar with Add to Cart and Buy Now, plus a wishlist heart and native share of a deep link.	✓ Every element of the product-detail mockup present and functional ✓ Variant change updates price and stock with no refetch ✓ Video does not load until tapped, so it costs nothing to scroll past
T2-18	MOB-1	Mobile · Crop Advisor intake	1. Intake form: crop name with autocomplete, issue description, affected area, and a preferred call window. 2. Media picker allowing up to 5 photos and 3 clips, with the caps enforced in the UI so the user is never allowed to exceed them and then rejected. 3. Compress before upload with react-native-compressor at 720p, show a per-file progress bar, and allow retry of a single failed file without restarting the whole ticket. 4. Confirmation screen with the ticket number, the SLA countdown, and honest copy: "An expert will call you before 11:15 AM." 5. Crop Advisory History list with status pills, and a detail view showing the expert's written advice and tappable recommended products.	✓ A ticket with 3 photos and a clip uploads over mobile data in under 40 seconds ✓ A 45 MB source video lands as roughly 3 MB in Cloudinary ✓ The farmer always knows their ticket number and when the call is due
T2-19	ALL	Friday gate — Week 2 demo	1. Demo the full farmer path on a device: home, search, product detail, and a real Advisor ticket with photos and video. 2. An expert works that ticket in the console and the written advice appears back in the app. 3. Read the Cloudinary credit meter aloud and extrapolate it to 30 days. 4. Confirm Razorpay test mode works end to end before Week 3 depends on it. 5. Decide on cut-list item #1 now if the recommendation feed cost more than a day.	✓ Gate passed only if a ticket goes phone → expert → written advice → back to phone ✓ Media budget confirmed sustainable with a real measured number ✓ Week 3 starts with commerce unblocked

§ 2.9 Week 3 — Commerce, Payments, Orders & Wallet

WEEK 3 · COMMERCE, PAYMENTS, ORDERS & WALLET

Focus: money moves correctly, or it does not move at all

This is the highest-risk week in the plan, because it is the only week where a bug costs rupees rather than time. Every task here is written around one idea: the server is the only thing that decides what something costs, what state an order is in, and how much is in a wallet. The client displays those decisions and nothing more.

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T3-01	BE-2	Commerce · Pricing engine	- One pure function — price(cart, coupon, walletIntent, address) → subtotal, discount, shipping, COD fee, wallet applied, total — used by cart, checkout and order creation. Three call sites, one implementation. - Free delivery above the threshold in app_config; flat shipping below it; COD fee only on COD. - All arithmetic in integer paise. Rounding is defined once, at the total, and documented in a comment that says why. - Return a line-by-line breakdown so the payment screen can render exactly what the mockup shows without doing any maths. - Unit-test 25 combinations including empty cart, coupon below minimum, wallet exceeding total, and free-delivery boundary at exactly the threshold.	✓ Cart, checkout and order totals are identical by construction ✓ Rupee-fraction drift is impossible because there are no fractions ✓ The breakdown the farmer sees is the number the gateway charges
T3-02	BE-2	Commerce · Coupon engine	- POST /cart/coupon validates code, active window, minimum order value, per-user limit and global limit, and returns the discount without applying anything yet. - Support FLAT, PERCENT with a discount cap, and REFERRAL kinds. One table, three behaviours. - Reserve the redemption only at order creation, inside the order transaction, so an abandoned checkout does not consume a limited coupon. - Release the redemption on cancellation or payment failure. - Return specific errors — expired, minimum not met, already used — not a generic "invalid coupon".	✓ Coupon apply and remove work on cart and payment screens ✓ A global limit of 100 cannot be exceeded by 20 parallel checkouts ✓ Cut-list item #5 can remove the engine while leaving referrals working
T3-03	BE-2	Orders · State machine core	- Implement assertTransition(from, to, actorRole) against the table in §2.4 as data, not as a chain of if-statements. - Make it the only writer of orders.status. Add a CI grep that fails the build on any other assignment to that column. - Every accepted transition writes an order_events row in the same transaction — timeline and status can never disagree. - Return 422 with the allowed set on an illegal transition, and 403 when the actor's role is wrong. - Write the guard test suite: CONFIRMED → DELIVERED is 422, a delivery agent cancelling is 403, DELIVERED is terminal, and every legal path succeeds.	✓ Guard suite green — this is the gate for the whole week ✓ Timeline is derived from events, never from a duplicated status field ✓ No code path can skip a state, including from the ops console

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T3-04	BE-2	Orders · Checkout & lifecycle	1. POST /orders in one transaction: revalidate prices, reserve stock with the conditional decrement, snapshot address and products as JSONB, reserve the coupon, create the order. 2. Fail loudly and specifically: 409 with the variant name on stock loss, 409 on a price change, 422 on an unserviceable pincode. 3. Ops transition endpoints for PACKED and DISPATCHED, each guarded by assertTransition and each writing an audit row. 4. The DELIVERED transition enqueues referral.credit and reviews.prompt — as queued jobs, so a notification failure can never roll back a delivery. 5. GET /orders and GET /orders/:no returning the full event timeline for the tracking screen.	✓ End-to-end order placed from the app and visible in the ops console ✓ Two farmers racing for the last unit: one order, one clean 409 ✓ Stock, coupon and order state are consistent after any failure
T3-05	BE-2	Money · Razorpay checkout & capture	1. Create the gateway order server-side with the amount from T3-01. The client never sends an amount. 2. Return the checkout options to the app; the app opens Razorpay Standard Checkout and reports back only for UX purposes. 3. Process payment.captured and payment.failed from the webhook queue, moving the order through assertTransition. The client callback is never trusted to confirm anything. 4. Finish payments.reconcile: for every order in PENDING_PAYMENT over 10 minutes, ask the gateway what actually happened and settle it. 5. Expire unpaid orders after 30 minutes, releasing stock and the coupon.	✓ Payment succeeds while the app is force-closed mid-transaction ✓ A dropped webhook is caught by reconciliation within 10 minutes ✓ No order is ever left in an indeterminate paid-or-not state
T3-06	BE-2	Commerce · Referral code & the ₹50 coupon	1. Generate referral_code at signup from the name plus 4 random characters, unique-checked with a retry. 2. Validate the code at signup: it must exist, be active, not be the referee's own, and the referee must not already have a referrer. 3. Auto-apply the ₹50 REFERRAL coupon on the referee's first checkout, subject to the minimum order value from config, and show it in the breakdown as a named line. 4. GET /referrals/me returns the code, a share message in the user's language, invited count, and how much has been credited so far. 5. Both amounts come from app_config. Marketing will change ₹50 and ₹100 and it must not be a deployment.	✓ Referee sees ₹50 off automatically on their first order ✓ A code cannot be self-applied or reused by the same referee ✓ Refer & Earn screen shows real numbers
T3-07	BE-2	Money · Wallet ledger & wallet payment	1. Implement credit() and debit() as the only two functions that touch the ledger, both requiring an idempotency_key and both updating the cached balance in the same transaction. 2. Set balance_after_paise on every row so any historical balance is readable without replaying the ledger. 3. Wallet payment is all-or-nothing: full balance covers the order or it is not used. No split tender. Re-read the balance inside the transaction and return 409 if it is short. 4. Refunds credit the wallet only — no gateway refunds in Phase 1, which removes an entire class of reconciliation work. 5. GET /wallet and GET /wallet/ledger for the balance, the History tab and the How-to-use copy.	✓ Double-submitting a refund credits exactly once ✓ Cached balance always equals the ledger sum — asserted by a nightly check job ✓ The append-only rule from T1-04 is proven by a failing UPDATE test

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T3-08	BE-1	Orders · Cancellation flow	1. POST /orders/:no/cancel allowed only from PENDING_PAYMENT, CONFIRMED and PACKED, with the reason codes from the mockup. 2. In one transaction: transition to CANCELLED, release stock, release the coupon redemption, credit the wallet keyed cancel:<order_id>, and notify. 3. Ops can cancel later with a mandatory reason, and it writes to audit_logs. 4. Cancelling a COD order refunds nothing, because nothing was collected — assert this explicitly in a test rather than relying on it being obvious. 5. This task is BE-1 working inside BE-2's module by design: it is the cross-training obligation from \$2.1, not a scheduling accident.	✓ Cancel works from the app and the console with correct wallet effects ✓ Double-tapping Cancel credits once ✓ BE-1 has hands-on familiarity with the orders module before Week 4
T3-09	BE-1	Platform · Order notifications & timeline	1. Wire all five order-status pushes plus payment success, payment failure, cancellation and wallet credit through the single notify() function from T1-10. 2. Write copy for every notification in all four languages, with deep links that open the specific order. 3. Dispatch from the job queue, never inline in the request — a slow FCM call must not slow down a checkout. 4. Build the timeline projection from order_events: five farmer-visible steps with timestamps, collapsing internal states the farmer should not see. 5. Handle the multi-device case: send to every active token, and deactivate the ones that fail.	✓ Status change produces a push within 30 seconds on a real device ✓ Tapping the push opens that exact order ✓ Timeline reads correctly in Hindi and Telugu
T3-10	BE-1	Advisor · Hardening & ops queue	1. Unassigned-ticket queue with manual assignment, for when no expert matches the language or district. 2. Expert availability windows, so the assignment algorithm stops routing tickets to someone who is off. 3. SLA metrics endpoint feeding the console: on-time percentage, median response, breach count, by expert and by day. 4. Reopen flow — a farmer can reopen a resolved ticket within 48 hours, which suspends the media purge. 5. Fire a Sentry alert on any breach so a missed callback is a visible incident and not a quiet row in a table.	✓ No ticket can silently sit unassigned ✓ SLA performance is a number reported at every Friday demo ✓ Breaches are noticed the same day they happen
T3-11	BE-2	Money · Referral credit job	1. referral.credit handler: reload the referral and the order, and confirm the order is still DELIVERED and not fully returned. 2. Credit ₹100 to the referrer keyed ref:<referral_id>, set the referral to CREDITED, and push "₹100 added to your wallet". 3. Enforce the monthly per-referrer cap from config; over the cap, mark the referral CAPPED with a reason rather than failing the job. 4. Reject when the referrer and referee share a phone number or a device ID. 5. Make the handler safe to run twice — the unique idempotency key is the safety net, but the guard clauses come first.	✓ Referrer receives ₹100 only after the referee's order is delivered ✓ Re-running the job ten times credits once ✓ A cancelled first order pays out nothing

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T3-12	BE-1	Platform · Security hardening & load test	1. Rate-limit every write endpoint by user and IP; strict limits on OTP, coupon apply, order creation and Advisor intake. 2. Add helmet, a CORS allowlist of exactly two origins, and body-size limits. 3. Configure Pino redaction for phone, OTP, token, Authorization header and gateway payloads — then read the deployed logs and confirm nothing leaked. 4. Verify object-level authorisation on every :id route: farmer A must get 404, not 403, on farmer B's order. 5. Run a k6 test at 50 concurrent users through browse, cart and checkout; record p95 latency and Cloud Run cold-start behaviour.	✓ An authorisation test suite covers every ownership-scoped route ✓ No personal data in logs, verified by reading them ✓ p95 under 500 ms warm, and cold-start latency measured rather than assumed
T3-13	FE-1	Console · Ops order board	1. Order list with status tabs, search by order number or phone, and date and payment filters. 2. Order detail: items with snapshots, the full event timeline, the payment record, and the address snapshot — clearly marked as the address at time of order. 3. Status action buttons that call the guarded transition endpoints and surface the 422 message when a step is skipped, so ops learns the FSM from the UI. 4. Bulk mark-as-packed for a day's orders, and a printable packing slip. 5. Cancellation and refund action with a mandatory reason, showing the wallet effect before it is confirmed.	✓ Ops can run a full day of fulfilment from this screen ✓ Illegal transitions are impossible from the UI and explained when attempted ✓ Every action traceable in the audit log
T3-14	FE-1	Console · Coupon, wallet & referral tools	1. Coupon CRUD with usage counters and a redemption list per coupon. 2. Wallet viewer per user showing the ledger with reasons and references, and an adjustment form requiring a reason and respecting the daily operator cap. 3. Referral leaderboard with a freeze action on a suspicious code, plus a same-phone flag. 4. Review moderation queue with publish, reject and bulk actions. 5. Reconciliation view listing orders whose payment status and order status disagree — the single screen that catches money bugs.	✓ Ops can resolve a payment dispute without a developer or a SQL client ✓ Every money adjustment has a name, a reason and a timestamp ✓ Reconciliation view is empty at the end of every day
T3-15	MOB-1	Mobile · Cart & coupon	1. Cart screen with quantity steppers, per-line remove with undo, and the server-computed breakdown rendered exactly as returned. 2. Coupon input with apply and remove, showing the specific server error rather than a generic failure. 3. "You might like this" strip below the cart, and the "other products" strip from the mockup. 4. Surface price-changed and stock-dropped flags inline on the affected line, with a one-tap update. 5. Empty-cart state that routes back into the catalog instead of dead-ending.	✓ Cart matches the mockup and every total comes from the server ✓ Coupon errors are specific and actionable ✓ A stale cart cannot be checked out silently at the old price

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T3-16	MOB-1	Mobile · Checkout & payment	1. Address selection with add-new inline, and the default address preselected. 2. Payment-method selection: Razorpay online, COD subject to the config ceiling, or Wallet when the balance covers the whole total — with an explanatory note when it does not. 3. Order-summary block showing subtotal, discount, shipping, COD fee and total, all from the pricing response. 4. Razorpay Standard Checkout in a WebView with correct handling of success, failure, dismissal and the app being backgrounded mid-payment. 5. Success screen driven by GET /orders/:no with a short poll, because the webhook is the source of truth — never mark an order paid from the client callback.	✓ Online, COD and wallet payments all complete end to end ✓ Backgrounding the app mid-payment still results in a correct order ✓ Failure offers a retry that does not create a duplicate order
T3-17	MOB-1	Mobile · Orders & tracking	1. Orders list with active and past tabs, thumbnails, status pills and the expected arrival date. 2. Order detail with the five-step vertical timeline built from order_events, each step timestamped. 3. Poll every 30 seconds only while the screen is focused and the order is active; otherwise rely on push. Battery and free-tier requests both matter. 4. Cancel action visible only in cancellable states, with the reason sheet and a clear statement of where the refund goes. 5. Reorder action that repopulates the cart, skipping anything now out of stock and saying so.	✓ Timeline updates within 30 seconds of an ops action ✓ Cancel is impossible to attempt in a non-cancellable state ✓ Arrival expectation is set honestly rather than optimistically
T3-18	MOB-1	Mobile · Profile hub & wallet	1. Profile hub with every entry from the mockup: Wishlist, Wallet, Saved Address, Crop Advisory History, Help & Support, Write to Us, Refer & Earn, Account Settings, Logout. 2. Wallet screen with balance, History list showing reason and reference, and the How-to-use tab. Add Money is present but disabled with "coming soon" copy — cut-list item #7. 3. Refer & Earn screen with the code, a native share sheet carrying a localised message, invited count and total earned. 4. Account Settings: language switch, notification toggles, terms, privacy policy, delete-account request, and log out everywhere. 5. Help & Support with FAQs from the API plus call and WhatsApp buttons, and Write to Us posting a ticket with optional photos.	✓ Every profile entry in the mockup is navigable and functional ✓ Wallet history explains where each rupee came from in plain language ✓ Language change takes effect immediately across the whole app
T3-19	ALL	Friday gate — Week 3 demo, the money gate	1. Place three real orders on the deployed system: one online, one COD, one paid from the wallet. 2. Cancel one and show the wallet credit landing with the correct reason and reference. 3. Complete a referral end to end and show the ₹50 discount and the ₹100 credit — the credit only after delivery. 4. Replay a webhook twenty times live and show exactly one state change. 5. Open the reconciliation view and confirm it is empty.	✓ Gate is binary: if any rupee is unexplained, the week is not done. ✓ Guard suite, idempotency tests and the reconciliation view all green ✓ Week 4 is delivery, polish and release — not commerce catch-up

§ 2.10 Week 4 — Delivery, Returns, Hardening & Release

WEEK 4 · DELIVERY, RETURNS, HARDENING & RELEASE

Focus: close the loop after delivery, then ship it to a real farmer

Week 4 closes the post-delivery loop, finishes the four languages, and puts the app in front of real farmers before it goes anywhere near a public listing. Reserve Thursday and Friday entirely for user testing and release — do not schedule features into them. Every project that skips this reservation ships on Monday instead.

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T4-01	BE-1	Orders · Shipment assignment & delivery OTP	1. Delivery-agent role with a lightweight login using the same OTP endpoints. 2. Assign an agent at DISPATCHED, generate a 6-digit delivery OTP, store only its hash, and send the plaintext to the farmer. 3. GET /agent/deliveries returns today's assignments with addresses, phone numbers, order values and COD amounts. 4. Agent transitions to OUT_FOR_DELIVERY, all through assertTransition with role checks. 5. Record the COD amount collected per delivery for the end-of-day cash reconciliation sheet.	✓ An agent can work a route from a browser on a phone ✓ Delivery OTP never stored or logged in plaintext ✓ COD cash is reconcilable at the end of every day
T4-02	BE-1	Platform · Four-language content pass	1. Extract every string from mobile and console into en/hi/te/kn and add a CI check that fails on a missing key in any locale. 2. Get translations reviewed by a native speaker. Machine-translated agricultural terminology is wrong in ways that matter — a mistranslated pesticide instruction is a safety problem, not a polish problem. 3. Localise all four database content surfaces: category names, FAQs, notification templates and error messages. 4. Format numbers, dates and currency per locale, and verify Devanagari, Telugu and Kannada scripts render without clipping in every component. 5. This is a staffing task as much as an engineering one. If reviewers are not lined up, ship English and Hindi only — that is cut-list item #8, and it is an honest cut.	✓ No hardcoded strings anywhere, enforced in CI ✓ Telugu and Kannada verified on a physical device, not an emulator ✓ Every shipped language has been read by a human who speaks it
T4-03	BE-1	Platform · Backups, restore drill & runbook	1. Confirm Neon's point-in-time recovery window and, on top of it, run a nightly pg_dump to Cloudflare R2 with 14 days of retention. 2. Perform an actual restore into a scratch Neon branch and time it. An untested backup is a belief, not a backup. 3. Write the incident runbook: API down, database at capacity, Cloudinary credits exhausted, payment webhooks failing, SLA breaches climbing, secret leaked. 4. Document the rollback procedure for API, console and the mobile OTA channel, and rehearse the API rollback once. 5. Set up UptimeRobot on /healthz, the console and the webhook endpoint, alerting to a shared channel rather than one person.	✓ Restore drill completed with a recorded recovery time ✓ Runbook is one page per scenario and readable at 2 a.m. ✓ Alerts reach more than one human

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T4-04	BE-1	Orders · Delivery confirmation & geofence	1. POST /agent/deliveries/:id/complete takes the delivery OTP, the agent's coordinates, and optional proof photos. 2. Verify the OTP hash, then compute the Haversine distance to the address lat/lng and require it to be within 200 m. 3. Store the measured distance_m regardless of outcome — it is the audit trail for a disputed delivery. 4. Outside the fence, allow an override with a mandatory reason that writes to audit_logs. Rural GPS drifts and a hard block would strand real deliveries. 5. On success: transition to DELIVERED, collect COD, and enqueue the review prompt for +24 h. Plain trigonometry — no PostGIS.	✓ Delivery cannot be marked complete from the wrong place without a logged reason ✓ Distance recorded on every delivery ✓ COD collection and the delivered state are set in one transaction
T4-05	BE-1	Platform · Free-tier audit & escape-hatch rehearsal	1. Write a single /internal/usage endpoint that reports Neon storage and compute hours, Cloudinary credits, Cloud Run invocations, WhatsApp conversations and Sentry events. 2. Add threshold alerts at the levels in the manager's \$1.5 and route them to the shared channel. 3. Rehearse the degradations: flip the Advisor video flag to one 10-second clip, and run the purge window at 7 days. Confirm both work before they are ever needed. 4. Write the Oracle Cloud always-free migration note: what to provision, what changes in the pipeline, and the honest two-day estimate. 5. Hand the tracking sheet and the alert channel to BE-1 as a named weekly duty rather than a collective intention.	✓ Every free-tier meter readable from one endpoint ✓ Degradation paths tested, not merely documented ✓ The escape hatch has a written procedure with a real time estimate
T4-06	BE-2	Orders · Returns with photo evidence	1. POST /orders/:no/items/:itemId/return within the return window from config, with reason codes and mandatory photo evidence through the signed-upload path. 2. Returns are per item and never change the order status — the order stays DELIVERED. A return is a refund event. 3. Ops adjudication: approve with a refund amount, or reject with a reason. Approval credits the wallet keyed return:<return_id>. 4. Restock only on an ops-confirmed physical receipt, never on approval — agricultural inputs that have been opened cannot go back on the shelf. 5. Notify the farmer at every state change. Cut-list item #6; the manual fallback is a support ticket and an ops goodwill credit.	✓ Return raised with photos from the app and adjudicated in the console ✓ Refund lands in the wallet exactly once ✓ Order history stays truthful — DELIVERED with a returned item
T4-07	BE-2	Catalog · Review write path & prompt	1. reviews.prompt job runs hourly, finds delivered items 24 hours old with no prompt sent, pushes once, and stamps review_prompt_sent_at so it never nags twice. 2. POST /reviews keyed on order_item_id, so only the buyer of that item can review it and only once — enforced by the unique index, not by application logic. 3. Rating 1-5 plus optional text. Update the rating rollup in the same transaction as the insert. 4. Route reviews containing flagged content to moderation instead of publishing them; publish the rest immediately. 5. Review photos are cut-list item #2 — text and stars only in Phase 1, which also keeps Cloudinary credits for the Advisor where they matter.	✓ Prompt arrives 24 hours after delivery, exactly once ✓ A user cannot review a product they did not buy, or review it twice ✓ Reviews appear on product detail with the histogram updating

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T4-08	BE-2	Money · Ledger integrity & refund audit	1. Nightly money.audit job: assert every wallet's cached balance equals the sum of its ledger, and that every balance_after_paise is internally consistent. 2. Assert every CREDIT has a valid reference — a cancellation, a return, a referral or a logged adjustment. An unexplained credit is a bug or a fraud. 3. Assert that no order has a captured payment without a CONFIRMED-or-later status, and no CONFIRMED order has an uncaptured non-COD payment. 4. Any failed assertion raises a Sentry alert and appears in the console reconciliation view. 5. Produce a one-page daily money summary: GMV, COD collected, wallet credited, refunds issued, referrals paid.	✓ Integrity job green on a database with several hundred seeded transactions ✓ Any drift is caught within 24 hours rather than at month end ✓ Ops has a daily money number without asking a developer
T4-09	BE-2	Platform · Test suite & contract conformance	1. Bring integration coverage on the money and order paths to a level the team will actually trust: checkout, payment, cancellation, return, wallet, referral. 2. Add contract conformance tests asserting every response validates against its Zod schema, so the OpenAPI document cannot drift from reality. 3. Add the concurrency suite: parallel checkout for the last unit, duplicate webhook, double refund, double referral credit. 4. Wire the whole suite into CI as a merge blocker, with a seeded ephemeral Neon branch per run. 5. Write the manual QA script for Thursday's UAT: 30 numbered steps covering every farmer-visible flow.	✓ CI blocks a merge that breaks a money path ✓ Contract and implementation proven identical ✓ UAT script exists and can be run by a non-developer
T4-10	FE-1	Console · KPI dashboard	1. Dashboard cards: orders today, GMV, average order value, COD share, new farmers, active carts. 2. Advisor panel: open tickets, on-time percentage, median response time, breaches this week, load per expert. 3. Free-tier panel reading /internal/usage, with amber past 60% and red past 80% — the numbers from the manager's \$1.5 rendered where people look daily. 4. Top products, top categories and top suppliers over a selectable range. 5. Everything from live queries; no scheduled reports and no second data store.	✓ One screen answers "how is Atavishaala doing today?" ✓ Free-tier risk is visible to the whole team without opening five vendor consoles ✓ Advisor SLA performance is impossible to overlook
T4-11	FE-1	Console · Returns adjudication & support inbox	1. Returns queue with the evidence photos shown large, the order context beside them, and approve or reject with a refund amount. 2. Restock toggle handled as a separate confirmation from refund approval, because they happen at different times. 3. Support inbox with status, assignment, canned replies in four languages, and the conversation thread. 4. Link every ticket to its order and farmer so ops has context without searching. 5. SLA colouring on support tickets — the same visual language as the Advisor board, so ops learns one pattern.	✓ A return is decided in under a minute with the evidence on screen ✓ No support ticket sits unassigned for more than a day ✓ Refund amounts always tie back to an order line

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T4-12	FE-1	Console · Delivery agent web app	1. A deliberately minimal mobile-web route for agents: OTP login, today's list, and one detail screen. No second native app. 2. Detail screen: address with a maps deep link, farmer phone with a tap-to-call, order items, COD amount in large type. 3. Mark out-for-delivery, then complete with the OTP field, geolocation capture and optional proof photo. 4. Show the geofence result clearly, and require the override reason when outside 200 m. 5. Make it work on a 3-year-old Android browser on a weak connection: no heavy bundle, no map SDK, large tap targets.	✓ An agent completes a delivery from a cheap phone in a village ✓ COD amount unmissable, reducing collection errors ✓ Zero additional app-store or build cost
T4-13	MOB-1	Mobile · Returns & review submission	1. Return request flow from order detail: item picker, reason sheet, mandatory photo capture with guidance copy, and confirmation. 2. Return status tracking with honest copy about when the refund reaches the wallet. 3. Review submission from the push deep link and from order detail: star selector, optional text, and a thank-you state. 4. Handle the already-reviewed case gracefully by showing the existing review rather than a 409. 5. Reviews are read-only after submission in Phase 1, and the UI says so before submitting.	✓ Return with photos submitted from a real device ✓ Review push opens straight into the rating screen for the right item ✓ No dead end anywhere in either flow
T4-14	MOB-1	Mobile · Notification inbox & deep links	1. Notification list from the API with unread badges and mark-as-read on open. 2. Register every deep link route and test all of them from a cold start, a warm start, and while the app is foregrounded. 3. Handle notification permission properly on Android 13+, asked at a moment that makes sense rather than on first launch. 4. In-app banner for pushes that arrive while the app is open, tappable to navigate. 5. Verify the Advisor callback notification, order status pushes and wallet credits all land and route correctly.	✓ Every notification type opens the correct screen from every app state ✓ Notification list works even for pushes the device never displayed ✓ Permission prompt appears in context and is accepted more often than not
T4-15	MOB-1	Mobile · Performance, accessibility & polish	1. Profile with Hermes and fix the worst offenders: convert lists to FlashList, memoise cards, and remove re-render storms on the home screen. 2. Test on a genuinely low-end Android device — 2 GB RAM, Android 11 — on a throttled connection. That is the real device, not the developer's phone. 3. Accessibility pass: labels on every touchable, minimum 44 pt targets, contrast checked against the brand palette, and font-scaling to 200% without clipping. 4. Add empty, loading and error states to every screen that still lacks one, and remove every remaining console log. 5. Trim the bundle: audit dependencies, drop anything unused, and confirm the APK size is reasonable for a rural download.	✓ Home screen interactive in under 2.5 s on the low-end device ✓ No screen can show a blank white body in any state ✓ Screen reader can complete a purchase

ID	DEV	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
T4-16	MOB-1	Mobile · Build, OTA channel & store track	1. Configure EAS Build profiles for development, preview and production, and produce a signed release build. Do this on Monday, not Friday — a first production build always surprises somebody. 2. Set up EAS Update channels so JavaScript-only fixes ship without a store review — this is the single most valuable operational lever the mobile app has. 3. Prepare the Play Store listing: screenshots in English and Hindi, description, privacy policy URL, data-safety declaration and content rating. 4. Publish to the internal testing track and get it installed on the whole team's phones plus the expert's. 5. Document the release procedure: what needs a store submission, what can go out over the air, and how to roll back either.	✓ Signed production build installed from the internal track ✓ An OTA fix demonstrated live, from commit to a phone in minutes ✓ Store listing complete and submitted for review, not started on release day
T4-17	ALL	Thursday — field UAT with real farmers	1. Run the T4-09 script with 5 to 10 actual farmers on their own devices and their own connections. Do not substitute the team for users. 2. Watch specifically for language comprehension, whether the Advisor flow is understood without explanation, and whether OTP arrives reliably on their networks. 3. Have one real expert work real tickets for the full day and time the callbacks honestly. 4. Log every issue as blocker, major or minor. Fix blockers Thursday night; majors are triaged, not assumed. 5. Write down what farmers asked for that is not in the app — that is the Phase 2 backlog, sourced from users rather than from us.	✓ Zero blockers remaining before the Friday gate ✓ Advisor SLA validated with real humans and a real clock ✓ A user-sourced Phase 2 list exists
T4-18	ALL	Friday gate — release readiness	1. Walk all 14 capabilities from the manager's §1.7 against observable criteria. Anything not demonstrable is not done. 2. Switch Razorpay to live keys and place one real ₹1 order end to end, then refund it. 3. Confirm backups, alerts, uptime monitors and the runbook are live, and that alerts reach more than one person. 4. Read the free-tier meters one final time and record the month-one baseline. 5. Agree the Phase 2 order, the on-call rotation for week one of production, and who watches the Advisor board on day one.	✓ Go / no-go decided against evidence rather than optimism ✓ Live payment proven with real money ✓ Someone is named as responsible for each of: uptime, money, and the SLA

APPENDIX · REFERENCE TABLES

Environment variables · API surface · free-tier limits · conventions

Appendix A · Environment Variable Checklist

Every variable below must exist in .env.example with a dummy value, in Google Secret Manager for the deployed API, and in GitHub Secrets for CI. A missing variable should fail at boot with a named error, never at first use in production — validate the whole environment with a Zod schema on startup.

VARIABLE	DEV	WHERE IT COMES FROM	NOTES
NODE_ENV · PORT · API_BASE_URL	BE-1	Set by Cloud Run and the pipeline	Cloud Run injects PORT — bind to it, do not hardcode 8080.
DATABASE_URL · DIRECT_URL	BE-1	Neon dashboard	Pooled URL for the app, direct URL for migrations. Mixing them up breaks Prisma migrate.
JWT_PRIVATE_KEY · JWT_PUBLIC_KEY	BE-1	Generated locally, stored in Secret Manager	RS256 PEM. Base64 the value to survive shell escaping in CI.
ACCESS_TOKEN_TTL · REFRESH_TOKEN_TTL	BE-1	Config decision	15m and 60d.
WHATSAPP_PHONE_NUMBER_ID · WHATSAPP_TOKEN · WHATSAPP_OTP_TEMPLATE	BE-2	Meta for Developers, after business verification	Primary OTP channel. Token needs periodic refresh.
SMS_API_KEY · SMS_SENDER_ID · SMS_DLT_TEMPLATE_ID	BE-2	2Factor.in plus TRAI DLT	Filed Day 1. Fallback channel only, and the only line item with a real per-unit cost.
RAZORPAY_KEY_ID · RAZORPAY_KEY_SECRET · RAZORPAY_WEBHOOK_SECRET	BE-2	Razorpay dashboard	Webhook secret is separate from the API secret. Keep test and live pairs clearly labelled.
CLOUDINARY_CLOUD_NAME · API_KEY · API_SECRET · UPLOAD_PRESET_*	BE-1	Cloudinary dashboard	Secret is used only to sign; it must never reach the client.
FCM_SERVICE_ACCOUNT_JSON	BE-1	Firebase project settings	Whole JSON as one base64 variable. Do not commit the file.
BREVO_API_KEY · SUPPORT_EMAIL	BE-2	Brevo dashboard	300 emails/day. Ops notifications only.
SENTRY_DSN_API · SENTRY_DSN_MOBILE · SENTRY_DSN_CONSOLE	BE-1	Sentry projects	Three separate projects so error volumes stay legible.
POSTHOG_KEY · POSTHOG_HOST	MOB-1	PostHog cloud	EU or US host must match the project region.
INTERNAL_JOB_SECRET	BE-1	Generated	Shared header for /internal/jobs/*. Rotate quarterly.
GCP_PROJECT_ID · GCP_SA_KEY · ARTIFACT_REGISTRY	BE-1	Google Cloud IAM	Deploy service account, least privilege — Cloud Run deploy and Artifact Registry write only.
R2_ACCOUNT_ID · R2_ACCESS_KEY · R2_SECRET · R2_BUCKET	BE-1	Cloudflare R2	Nightly pg_dump target.
EXPO_TOKEN · EAS_PROJECT_ID	MOB-1	Expo dashboard	For EAS Build and Update from CI.
EXPO_PUBLIC_API_URL · EXPO_PUBLIC_RAZORPAY_KEY_ID	MOB-1	Set per build profile	EXPO_PUBLIC_ variables are shipped in the bundle — public values only.
VITE_API_URL	FE-1	Set per Pages environment	Same rule: bundled, so nothing secret.

Appendix B · API Surface at Freeze

The complete Phase 1 surface, roughly 70 endpoints. This is what gets frozen at end of Day 3, Week 1. Anything not on this list is either Phase 2 or a contract change that needs a pull request and a note in the Wednesday sync.

GROUP	ENDPOINTS	AUTH	DEV	WEEK
Auth	POST /auth/otp/request · /auth/otp/verify · /auth/refresh · /auth/logout · GET /.well-known/jwks	Public	BE-1	W1
Me & addresses	GET PATCH /me · GET POST /addresses · PATCH DELETE /addresses/:id · POST /addresses/:id/default · POST /devices	Farmer	BE-1	W1
Catalog	GET /home · /categories · /suppliers · /banners · /products · /products/:slug · /products/:id/reviews · /search	Public	BE-1	W2
Media	POST /media/sign · POST /media/confirm	Any role	BE-1	W2
Advisor	POST GET /advisor/consultations · GET /advisor/consultations/:id · POST /advisor/consultations/:id/media · POST /:id/reopen	Farmer	BE-1	W2
Expert	GET /expert/queue · GET /expert/consultations/:id · POST /:id/notes · POST /:id/resolve · POST /:id/reassign	Expert	BE-1	W2-W3
Cart & wishlist	GET /cart · POST PATCH DELETE /cart/items · POST DELETE /cart/coupon · GET POST DELETE /wishlist	Farmer	BE-2	W2-W3
Orders	POST /orders · GET /orders · GET /orders/:no · POST /orders/:no/cancel · POST /orders/:no/items/:id/return · POST /orders/:no/reorder	Farmer	BE-2	W3-W4
Payments	POST /payments/create · POST /payments/webhook · GET /payments/:orderNo/status	Farmer / signed	BE-2	W3
Wallet & referral	GET /wallet · GET /wallet/ledger · GET /referrals/me · POST /referrals/validate	Farmer	BE-2	W3
Reviews	POST /reviews · GET /reviews/pending	Farmer	BE-2	W4
Support	GET /faqs · POST GET /support/tickets · GET /support/tickets/:id · POST /feedback	Farmer	BE-2	W2
Notifications	GET /notifications · POST /notifications/:id/read · POST /notifications/read-all	Farmer	BE-1	W3
Agent	GET /agent/deliveries · POST /agent/deliveries/:id/out · POST /:id/complete	Agent	BE-1	W4
Admin — catalog	Full CRUD on suppliers, categories, products, variants, media, banners · POST /admin/products/import	Ops / admin	BE-1	W2
Admin — ops	GET /admin/orders · POST /admin/orders/:no/status · /cancel · GET POST /admin/returns/:id · GET /admin/support/*	Ops	BE-2	W3-W4
Admin — money	GET POST /admin/coupons · GET /admin/wallets/:userId · POST /admin/wallets/:userId/adjust · GET /admin/referrals · GET /admin/reconciliation	Admin	BE-2	W3
Admin — platform	GET PATCH /admin/config · GET /admin/audit · GET POST /admin/experts · GET /admin/kpi · GET /admin/advisor/sla	Admin	BE-1	W1-W4
Internal	POST /internal/jobs/:name · GET /internal/usage · GET /healthz	Shared secret	BE-1	W1

Appendix C · Free-Tier Limits, and the Line You Must Not Cross

Learn the numbers in the fourth column. They are the ones that turn a ₹0 architecture into a bill. Every entry here is verified as of August 2026 — free tiers change, so re-check before Phase 2 rather than trusting this page a year from now.

SERVICE	FREE ALLOWANCE	WHAT WE EXPECT AT MVP	THE ENGINEERING RULE IT IMPLIES
Google Cloud Run	2M requests, 360k GB-s, 180k vCPU-s per month, always free	~200k requests	Never proxy media bytes, and keep responses small. A cold start is acceptable; a 40 MB upload through the API is not.
Neon Postgres	0.5 GB storage, 190 compute-hours per month	40–60 MB, well inside compute	Store references, not blobs. No base64 columns, no logs in the database, and purge otp_codes and finished jobs nightly.
Cloudinary	25 credits per month (≈25 GB storage or delivery)	~0.6 GB at 150 tickets	The single tightest constraint. Compress on the client, cap duration, never transform video, and honour purge_after.
WhatsApp Cloud API	1,000 service conversations per month	~600 OTP plus Advisor messages	One OTP per request — resend must reuse the same code within its window rather than sending a second conversation.
2Factor.in SMS	Trial credits only, then paid	Fallback for ~150 messages	The only real per-unit cost in the stack. WhatsApp first, always; SMS only on failure.
Firebase FCM	Unlimited, free	All push traffic	No limit to respect, but deactivate dead tokens so failure rates stay meaningful.
Cloudflare Pages	500 builds per month, unlimited requests, commercial use allowed	~80 builds	Two consoles on one project. This is why we are not on Vercel Hobby, which forbids commercial use.
Cloudflare DNS, SSL, WAF, CDN	Free plan, unlimited bandwidth	All edge traffic	Put catalog reads behind cache headers and let the edge absorb the repeats.
GitHub Actions	2,000 minutes per month on private repos	~900 minutes	Cron jobs are cheap; CI is not. Cache pnpm and Docker layers, and do not run the full suite on every push to a branch.
Sentry	5,000 errors per month	Under 1,000 if the app is healthy	Sample noisy errors and fix the top three each week, or a single loop consumes the month.
PostHog	1M events per month	~200k	Track deliberate events, not every render.
UptimeRobot	50 monitors, 5-minute interval	4 monitors	Enough for API, console, webhook and the marketing page.
Brevo email	300 per day	~30	Ops notifications only. Never send farmer-facing email in Phase 1 — farmers do not read it.
Expo EAS	30 builds per month on the free plan	~20	Build deliberately. Use EAS Update for JavaScript fixes instead of burning a build.
Razorpay	No monthly fee	2% plus GST per transaction	Not a free-tier limit — a real variable cost on GMV. It scales with revenue, which is the only acceptable kind of cost.

Appendix D · Working Conventions

AREA	THE CONVENTION	WHY IT IS WORTH THE FRICTION
Branching	main is always deployable. Work on feat/<module>-<short> or fix/<module>-<short> and squash-merge. No long-lived develop branch.	With four people and four weeks, a release branch is pure overhead. Trunk-based keeps integration pain spread across the sprint instead of concentrated in Week 4.
Pull requests	Reviewed by the module's reviewer from §2.1. Under 400 changed lines where possible. CI green is a hard gate, not a suggestion.	The reviewer column exists so knowledge is shared before someone is unavailable, not after.
Commits	Conventional commits: feat(orders): ..., fix(money): Reference the task ID.	The git log becomes the progress report, which means nobody has to write one.
Migrations	Every schema change is a committed Prisma migration, tested on the Neon preview branch first. Never edit a table by hand.	A hand-edited production table is undetectable until it breaks a deploy on a Friday.
Definition of done	Merged to main, deployed, contract updated, tests passing, expected outputs from the task table demonstrable on deployed infrastructure.	"Done on my machine" is the most expensive phrase in a four-week sprint.
Error handling	Every endpoint returns the agreed envelope. Never leak a stack trace or a database error to a client. Log the cause, return the code.	The mobile app can only show useful copy if the error codes are stable and enumerated.
Logging	Structured Pino with a request ID. Redact phone, OTP, token and gateway payloads. No console.log in merged code.	Logs get read by more people than you expect, and a phone number in a log is a data incident.
Testing	Unit tests for pricing, SLA computation and the FSM table. Integration tests for every money path. Concurrency tests for stock, webhooks, refunds and referrals.	These are the four places where a bug is measured in rupees rather than in bug reports.
Config over code	Any number a business person might change lives in app_config: referral amounts, COD ceiling, delivery threshold, SLA hours, feature flags.	Marketing will change ₹50 to ₹75 during the launch week. That must not be a deployment.
Asking for help	Blocked for more than 60 minutes, ask in the channel. Blocked for a day, it goes on the Wednesday sync agenda.	In a 20-day sprint, a day lost to pride is 5% of the project.

IF YOU READ ONLY ONE PAGE, READ §2.2

The twelve rules are not style preferences. Each one is a specific failure that has cost a real project real money: fractional rupees, mutated ledgers, replayed webhooks, client-trusted prices, in-memory state on a scale-to-zero host.

IF SOMETHING HAS TO GIVE, CUT IN ORDER

The manager's §1.6 lists eight cuts in a pre-agreed sequence. Take them from the top. Do not invent a new cut, and never quietly reduce the Advisor to save a week — it is the reason the product exists.

BUILD THE ADVISOR PROPERLY. THE REST IS A STOREFRONT.

Atavishaala · Farm Market · Team Execution Handbook · Phase 1 MVP · Companion volume: Manager's Strategic Roadmap